

Baking Recipe Keeper

Software Design Document

Android application · Kotlin + Jetpack Compose · Version 1.0 · September 2026

1. Overview

Baking Recipe Keeper is a native Android application for storing and managing baking recipes. Each recipe holds descriptive metadata, a list of ingredients (with amounts and units), and an ordered baking procedure. The app supports live serving-size scaling, search and filtering, sharing, and light/dark theming. All data is stored locally on the device using a relational database, and the interface is built entirely with a declarative UI toolkit.

2. Objectives & Scope

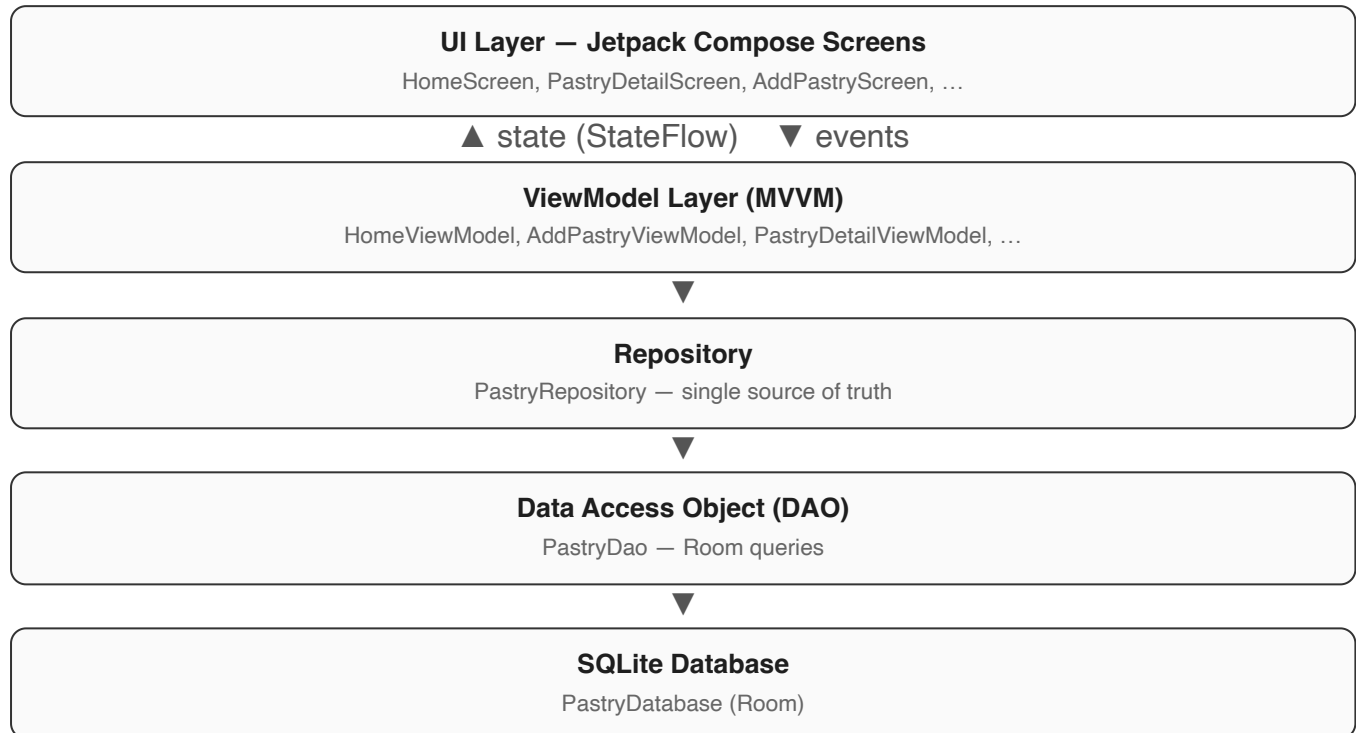
- Create, view, edit, and delete recipes with ingredients and step-by-step instructions.
- Scale ingredient quantities automatically based on desired servings.
- Provide a dashboard with counts, search, and category filtering.
- Aggregate ingredients across all recipes for an at-a-glance overview.
- Persist data reliably across app updates via database migrations.
- Offline-first: no network or account is required.

3. Technology Stack

Concern	Technology
Language	Kotlin
UI	Jetpack Compose, Material 3
Architecture	MVVM (Model–View–ViewModel)
Persistence	Room (SQLite) with schema migrations
Navigation	Navigation Compose
Async / state	Kotlin Coroutines & Flow (<code>StateFlow</code>)
Images	Coil (Compose)
Testing	JUnit (unit tests for core logic)
Min / Target SDK	API 26 (Android 8.0) / API 36

4. High-Level Architecture

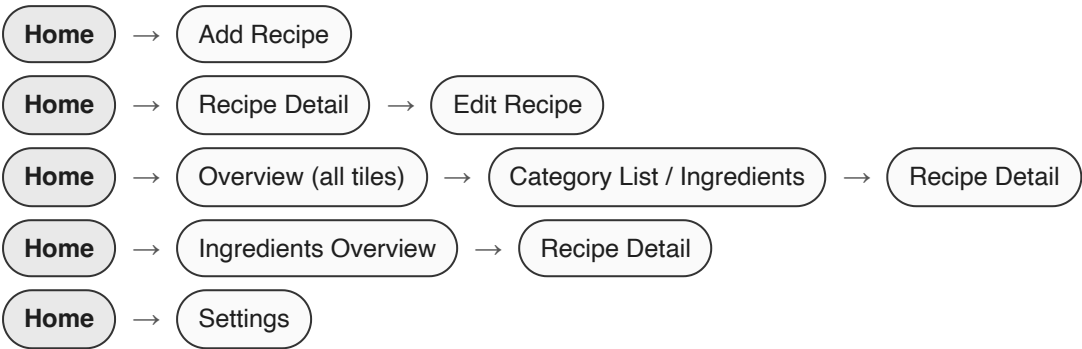
The app follows a unidirectional MVVM pattern. Compose screens observe immutable UI state exposed by ViewModels; ViewModels invoke a repository, which is the single gateway to the Room database. Data flows up as reactive streams; user events flow down as function calls.



The repository exposes recipes as a **Flow**; when the database changes, the UI updates automatically. Dependencies are supplied through ViewModel factories.

5. Navigation Flow

Screens are wired through a single `NavHost` . Routes and transitions:



Route	Screen	Purpose
home	HomeScreen	Dashboard, search, category filter, recipe list, add button.
add_pastry	AddPastryScreen	Create a new recipe.
edit_pastry/{id}	AddPastryScreen	Edit an existing recipe (same screen in edit mode).
detail/{id}	PastryDetailScreen	View a recipe, scale servings, share, edit, delete.
overview	OverviewScreen	Grid of all dashboard tiles.
ingredients	IngredientsListScreen	All ingredients grouped and searchable.
pastry_list/{category}	PastryListScreen	Recipes in a category (or all).
settings	SettingsScreen	Theme selection and app info.

6. Data Model

Three related entities. A recipe (`Pastry`) owns many ingredients and steps; deleting a recipe cascades to its children.



Pastry

Field	Type	Purpose
id	Int (PK)	Auto-generated primary key.
name	String	Recipe name (required).
category	String	One of the predefined categories.
description	String	Short description.
imageUri	String?	Path to the stored photo (nullable).
servings	Int	Base yield; drives scaling (0 = unset).
prepMinutes / cookMinutes	Int	Prep and bake time in minutes.
difficulty	String	Easy / Medium / Hard (optional).
createdAt / updatedAt	Long	Timestamps.

Ingredient

Field	Type
id	Int (PK)
pastryId	Int (FK)
name	String
amount	Double
unit	IngredientUnit

Step

Field	Type
id	Int (PK)
pastryId	Int (FK)
position	Int
instruction	String

IngredientUnit is an enum: Pieces, Grams, Kilograms, Ounces, Pounds, Milliliters, Liters, Teaspoons, Tablespoons, Cups, Pinch. Stored via a Room type converter.

PastryWithIngredients is a relation object that loads a recipe together with its ingredients and steps in one query.

7. Database & Migrations

`PastryDatabase` is the Room database (current version 5). Migrations preserve user data across updates:

Migration	Change
1 → 2	Added the <code>ingredients</code> table with a cascade foreign key.
2 → 3	Added <code>createdAt</code> / <code>updatedAt</code> to recipes.
3 → 4	Added the <code>steps</code> table.
4 → 5	Added servings, prep/bake minutes, and difficulty.

8. Function & Class Reference

8.1 Data Layer

Class	Member	Purpose
PastryDao	insertPastry()	Insert a recipe; returns the new row id.
PastryDao	updatePastry()	Update an existing recipe.
PastryDao	insertIngredients() / insertSteps()	Insert child rows for a recipe.
PastryDao	deleteIngredientsForPastry() / deleteStepsForPastry()	Clear children before re-inserting on edit.
PastryDao	deletePastry()	Delete a recipe (children cascade).
PastryDao	getAllPastriesWithIngredients()	Reactive list of all recipes with children.
PastryDao	getPastryWithIngredients(id)	Reactive single recipe with children.
PastryCategories	all / singular()	Predefined categories; singular form for counts.
Ingredient.kt	formatAmount()	Formats a Double amount without trailing ".0".
ImageStorage	deleteIfLocal()	Deletes a photo copied into app storage.
Converters	fromUnit()/toUnit()	Room type converter for the unit enum.

8.2 Repository

Member	Purpose
allPastries	Flow of every recipe (with ingredients & steps).
getPastryById(id)	Flow of one recipe.
insertPastryWithIngredients(...)	Save a new recipe plus its ingredients and steps.

<code>updatePastryWithIngredients(...)</code>	Update a recipe and replace its children.
<code>deletePastry(pastry)</code>	Remove a recipe.

8.3 ViewModels

Class	Responsibility
<code>HomeViewModel</code>	Builds <code>HomeUiState</code> : filtered list, search query, selected category, dashboard tiles, loading flag.
<code>AddPastryViewModel</code>	Holds the add/edit form state; validates the name and ingredient amounts; copies the photo into storage; inserts or updates; manages ingredient/step drafts and step reordering; detects unsaved changes.
<code>PastryDetailViewModel</code>	Exposes a single recipe as state; deletes it (and cleans up its image).
<code>OverviewViewModel</code>	Produces the full list of dashboard tiles.
<code>IngredientsListViewModel</code>	Groups ingredients across recipes; applies search.
<code>PastryListViewModel</code>	Filters recipes by category (or all); sorts alphabetically.
<code>buildDashboardTiles()</code>	Pure function turning recipes into count tiles (Ingredients, All Recipes, per-category).
<code>groupIngredients()</code>	Pure function that merges ingredients by name and sums per unit.

8.4 UI Screens

Screen	Responsibility
<code>HomeScreen</code>	Dashboard tiles, search bar, category chips, recipe cards, add FAB, settings entry, snackbars.
<code>AddPastryScreen</code>	Form for name, category, details, photo, ingredients, and baking procedure; drag-to-reorder dialog; discard-changes guard.
<code>PastryDetailScreen</code>	Displays a recipe; servings stepper scales amounts; share; full-screen photo; edit / delete.
<code>IngredientsListScreen</code>	Grouped, searchable ingredient list; taps open the owning recipe.
<code>PastryListScreen</code>	List of recipes for a category or all recipes.
<code>OverviewScreen</code>	Full grid of dashboard tiles.
<code>SettingsScreen</code>	Theme (System/Light/Dark) and app version.

9. Key User Flows

Add / Edit a recipe

1. User fills name, category, details, and adds ingredient and step rows.
2. ViewModel validates (name required; ingredient amounts > 0).
3. The photo is copied into app storage; the recipe, ingredients, and steps are saved.
4. The screen returns and a confirmation snackbar appears.

Scale servings

1. On the detail screen the user changes the servings stepper.
2. A scale factor is derived and every ingredient amount is recomputed for display.

Reorder procedure

1. User opens the reorder screen and long-presses a step card to pick it up.
2. The card lifts and follows the finger; steps swap and renumber live.

Delete a recipe

1. User confirms in a dialog (destructive action in red).
2. The recipe and its image are removed; a snackbar confirms on the previous screen.

10. Testing

Pure business logic is covered by JUnit unit tests:

Test	Verifies
DashboardTest	Tile counts and singular/plural category labels.
IngredientsGroupingTest	Case-insensitive merge, per-unit totals, sorting.
PastryCategoriesTest	Singularization of category names.
FormatAmountTest	Whole vs. fractional amount formatting.

11. Future Enhancements

- Favorites, duplicate recipe, and a shopping list.
- Cloud sync and multi-device backup (accounts).
- Guided “Start Baking” mode with per-step timers.
- Unit conversion and recipe import from the web.
- Subscription (freemium) with premium features.